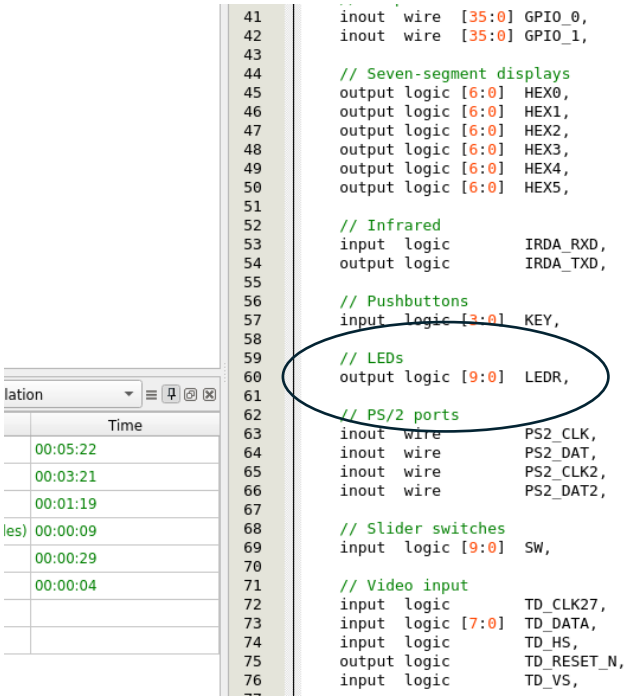
Starting from our top level module, we see that we have defined a 10 bit wide signal by name of LEDR. Not shown here, but in Assignment editor, LEDR[0] to LED[8] are assigned to their corresponding pins on the DE1-SoC board.

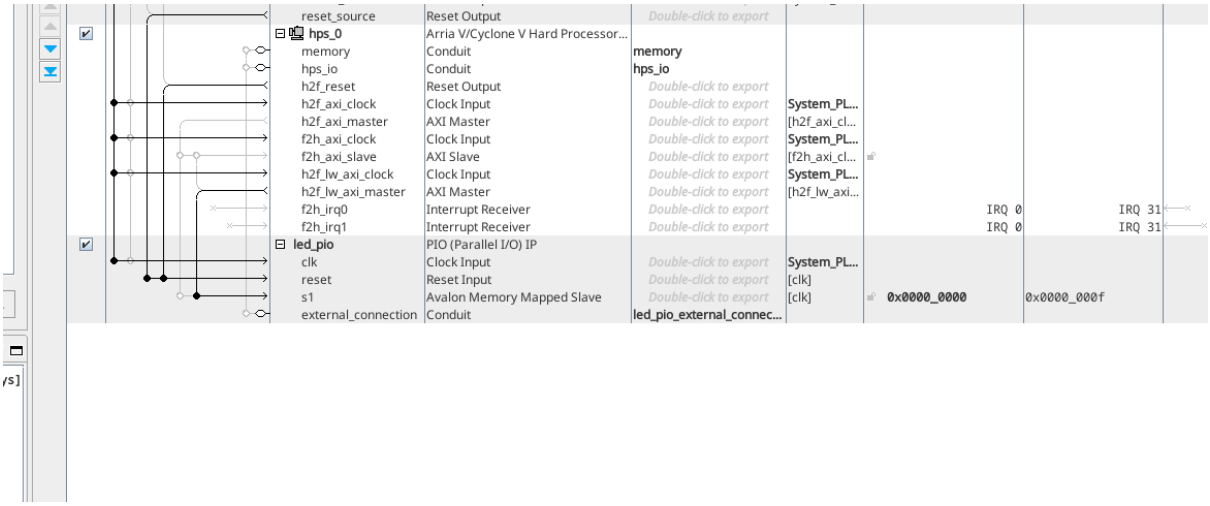
Within the top level module, we also see that we have created an instance of the platform designer module, and tied the LEDR signal to it's internal signal called led_pio_external_connection_export.




```
41      inout  wire  [35:0] GPIO_0,
42      inout  wire  [35:0] GPIO_1,
43
44      // Seven-segment displays
45      output logic [6:0]  HEX0,
46      output logic [6:0]  HEX1,
47      output logic [6:0]  HEX2,
48      output logic [6:0]  HEX3,
49      output logic [6:0]  HEX4,
50      output logic [6:0]  HEX5,
51
52      // Infrared
53      input  logic        IRDA_RXD,
54      output logic        IRDA_TXD,
55
56      // Pushbuttons
57      input  logic [3:0]  KEY,
58
59      // LEDs
60      output logic [9:0]  LEDR,
61
62      // PS/2 ports
63      inout  wire         PS2_CLK,
64      inout  wire         PS2_DAT,
65      inout  wire         PS2_CLK2,
66      inout  wire         PS2_DAT2,
67
68      // Slider switches
69      input  logic [9:0]  SW,
70
71      // Video input
72      input  logic        TD_CLK27,
73      input  logic [7:0]  TD_DATA,
74      input  logic        TD_HS,
75      output logic        TD_RESET_N,
76      input  logic        TD_VS,
```

```
    // LED
    .hps_io_hps_io_gpio_inst_GPIO53     (HPS_LED),

    // pushbutton
    .hps_io_hps_io_gpio_inst_GPIO54     (HPS_KEY),

    // Accelerometer
    .hps_io_hps_io_gpio_inst_GPIO61     (HPS_GSENSOR_INT),

    // DDR3 SRAM
    .memory_mem_a        (HPS_DDR3_ADDR),
    .memory_mem_ba       (HPS_DDR3_BA),
    .memory_mem_ck       (HPS_DDR3_CK_P),
    .memory_mem_ck_n     (HPS_DDR3_CK_N),
    .memory_mem_cke      (HPS_DDR3_CKE),
    .memory_mem_cs_n     (HPS_DDR3_CS_N),
    .memory_mem_ras_n    (HPS_DDR3_RAS_N),
    .memory_mem_cas_n    (HPS_DDR3_CAS_N),
    .memory_mem_we_n     (HPS_DDR3_WE_N),
    .memory_mem_reset_n  (HPS_DDR3_RESET_N),
    .memory_mem_dq       (HPS_DDR3_DQ),
    .memory_mem_dqs      (HPS_DDR3_DQS_P),
    .memory_mem_dqs_n    (HPS_DDR3_DQS_N),
    .memory_mem_odt      (HPS_DDR3_ODT),
    .memory_mem_dm       (HPS_DDR3_DM),
    .memory_oct_rzqin    (HPS_DDR3_RZQ),

    ///// FPGA SIDE /////

    .system_pll_ref_clk_clk          (CLOCK_50),             //          system_pll_ref_clk.clk
    .system_pll_ref_reset_reset      (1'b0),         //        system_pll_ref_reset.reset

    .led_pio_external_connection_export (LEDR)  // led_pio_external_connection.export
);
```

This screenshot from platform designer shows that we have exported the conduit from the LED_PIO resulting in the led_pio_external_connection_export that we can use.



To see how the signal works internally we need to open the generated HDL code. This screenshot shows the name of the module is platform_designer module and it has an output wire 10 bits wide called led_pio_external_connection_export.

```
`timescale 1 ps / 1 ps
module platform_designer_module (
		output wire        hps_io_hps_io_emac1_inst_TX_CLK,    //                    hps_io.hps_io_emac1_inst_TX_CLK
		output wire        hps_io_hps_io_emac1_inst_TXD0,      //                          .hps_io_emac1_inst_TXD0
		output wire        hps_io_hps_io_emac1_inst_TXD1,      //                          .hps_io_emac1_inst_TXD1
		output wire        hps_io_hps_io_emac1_inst_TXD2,      //                          .hps_io_emac1_inst_TXD2
		output wire        hps_io_hps_io_emac1_inst_TXD3,      //                          .hps_io_emac1_inst_TXD3
		input  wire        hps_io_hps_io_emac1_inst_RXD0,      //                          .hps_io_emac1_inst_RXD0
		inout  wire        hps_io_hps_io_emac1_inst_MDIO,      //                          .hps_io_emac1_inst_MDIO
		output wire        hps_io_hps_io_emac1_inst_MDC,       //                          .hps_io_emac1_inst_MDC
		input  wire        hps_io_hps_io_emac1_inst_RX_CTL,    //                          .hps_io_emac1_inst_RX_CTL
		output wire        hps_io_hps_io_emac1_inst_TX_CTL,    //                          .hps_io_emac1_inst_TX_CTL
		input  wire        hps_io_hps_io_emac1_inst_RX_CLK,    //                          .hps_io_emac1_inst_RX_CLK
		input  wire        hps_io_hps_io_emac1_inst_RXD1,      //                          .hps_io_emac1_inst_RXD1
		input  wire        hps_io_hps_io_emac1_inst_RXD2,      //                          .hps_io_emac1_inst_RXD2
		input  wire        hps_io_hps_io_emac1_inst_RXD3,      //                          .hps_io_emac1_inst_RXD3
		inout  wire        hps_io_hps_io_qspi_inst_IO0,        //                          .hps_io_qspi_inst_IO0
		inout  wire        hps_io_hps_io_qspi_inst_IO1,        //                          .hps_io_qspi_inst_IO1
		inout  wire        hps_io_hps_io_qspi_inst_IO2,        //                          .hps_io_qspi_inst_IO2
		inout  wire        hps_io_hps_io_qspi_inst_IO3,        //                          .hps_io_qspi_inst_IO3
		output wire        hps_io_hps_io_qspi_inst_SS0,        //                          .hps_io_qspi_inst_SS0
		output wire        hps_io_hps_io_qspi_inst_CLK,        //                          .hps_io_qspi_inst_CLK
		inout  wire        hps_io_hps_io_sdio_inst_CMD,        //                          .hps_io_sdio_inst_CMD
		inout  wire        hps_io_hps_io_sdio_inst_D0,         //                          .hps_io_sdio_inst_D0
		inout  wire        hps_io_hps_io_sdio_inst_D1,         //                          .hps_io_sdio_inst_D1
		output wire        hps_io_hps_io_sdio_inst_CLK,        //                          .hps_io_sdio_inst_CLK
		inout  wire        hps_io_hps_io_sdio_inst_D2,         //                          .hps_io_sdio_inst_D2
		inout  wire        hps_io_hps_io_sdio_inst_D3,         //                          .hps_io_sdio_inst_D3
		inout  wire        hps_io_hps_io_usb1_inst_D0,         //                          .hps_io_usb1_inst_D0
		inout  wire        hps_io_hps_io_usb1_inst_D1,         //                          .hps_io_usb1_inst_D1
		inout  wire        hps_io_hps_io_usb1_inst_D2,         //                          .hps_io_usb1_inst_D2
		inout  wire        hps_io_hps_io_usb1_inst_D3,         //                          .hps_io_usb1_inst_D3
		inout  wire        hps_io_hps_io_usb1_inst_D4,         //                          .hps_io_usb1_inst_D4
		inout  wire        hps_io_hps_io_usb1_inst_D5,         //                          .hps_io_usb1_inst_D5
		inout  wire        hps_io_hps_io_usb1_inst_D6,         //                          .hps_io_usb1_inst_D6
		inout  wire        hps_io_hps_io_usb1_inst_D7,         //                          .hps_io_usb1_inst_D7
		input  wire        hps_io_hps_io_usb1_inst_CLK,        //                          .hps_io_usb1_inst_CLK
		output wire        hps_io_hps_io_usb1_inst_STP,        //                          .hps_io_usb1_inst_STP
		input  wire        hps_io_hps_io_usb1_inst_DIR,        //                          .hps_io_usb1_inst_DIR
		input  wire        hps_io_hps_io_usb1_inst_NXT,        //                          .hps_io_usb1_inst_NXT
		output wire        hps_io_hps_io_spim1_inst_CLK,       //                          .hps_io_spim1_inst_CLK
		output wire        hps_io_hps_io_spim1_inst_MOSI,      //                          .hps_io_spim1_inst_MOSI
		input  wire        hps_io_hps_io_spim1_inst_MISO,      //                          .hps_io_spim1_inst_MISO
		output wire        hps_io_hps_io_spim1_inst_SS0,       //                          .hps_io_spim1_inst_SS0
		input  wire        hps_io_hps_io_uart0_inst_RX,        //                          .hps_io_uart0_inst_RX
		output wire        hps_io_hps_io_uart0_inst_TX,        //                          .hps_io_uart0_inst_TX
		inout  wire        hps_io_hps_io_i2c0_inst_SDA,        //                          .hps_io_i2c0_inst_SDA
		inout  wire        hps_io_hps_io_i2c0_inst_SCL,        //                          .hps_io_i2c0_inst_SCL
		inout  wire        hps_io_hps_io_i2c1_inst_SDA,        //                          .hps_io_i2c1_inst_SDA
		inout  wire        hps_io_hps_io_i2c1_inst_SCL,        //                          .hps_io_i2c1_inst_SCL
		inout  wire        hps_io_hps_io_gpio_inst_GPIO09,     //                          .hps_io_gpio_inst_GPIO09
		inout  wire        hps_io_hps_io_gpio_inst_GPIO35,     //                          .hps_io_gpio_inst_GPIO35
		inout  wire        hps_io_hps_io_gpio_inst_GPIO40,     //                          .hps_io_gpio_inst_GPIO40
		inout  wire        hps_io_hps_io_gpio_inst_GPIO41,     //                          .hps_io_gpio_inst_GPIO41
		inout  wire        hps_io_hps_io_gpio_inst_GPIO48,     //                          .hps_io_gpio_inst_GPIO48
		inout  wire        hps_io_hps_io_gpio_inst_GPIO53,     //                          .hps_io_gpio_inst_GPIO53
		inout  wire        hps_io_hps_io_gpio_inst_GPIO54,     //                          .hps_io_gpio_inst_GPIO54
		inout  wire        hps_io_hps_io_gpio_inst_GPIO61,     //                          .hps_io_gpio_inst_GPIO61
		output wire [9:0]  led_pio_external_connection_export, // led_pio_external_connection.export
		output wire [14:0] memory_mem_a,                       //                      memory.mem_a
		output wire [2:0]  memory_mem_ba,                      //                            .mem_ba
```

Upon scrolling down to see where the signal is used, we see it is defined within a submodule.

```
platform_designer_module_led_pio led_pio (
        .clk        (system_pll_sys_clk_clk),                  //                clk.clk
        .reset_n    (~rst_controller_reset_out_reset),         //              reset.reset_n
        .address    (mm_interconnect_0_led_pio_s1_address),    //                 s1.address
        .write_n    (~mm_interconnect_0_led_pio_s1_write),     //                   .write_n
        .writedata  (mm_interconnect_0_led_pio_s1_writedata),  //                   .writedata
        .chipselect (mm_interconnect_0_led_pio_s1_chipselect), //                   .chipselect
        .readdata   (mm_interconnect_0_led_pio_s1_readdata),   //                   .readdata
        .out_port   (led_pio_external_connection_export)       // external_connection.export
);
```

Looking at this final module we can see exactly the logic that has been implemented. Ultimately the main important part is that if the chipselect is high, write is enabled, and the address of this port is selected, the data_out register takes the value coming from our HPS.

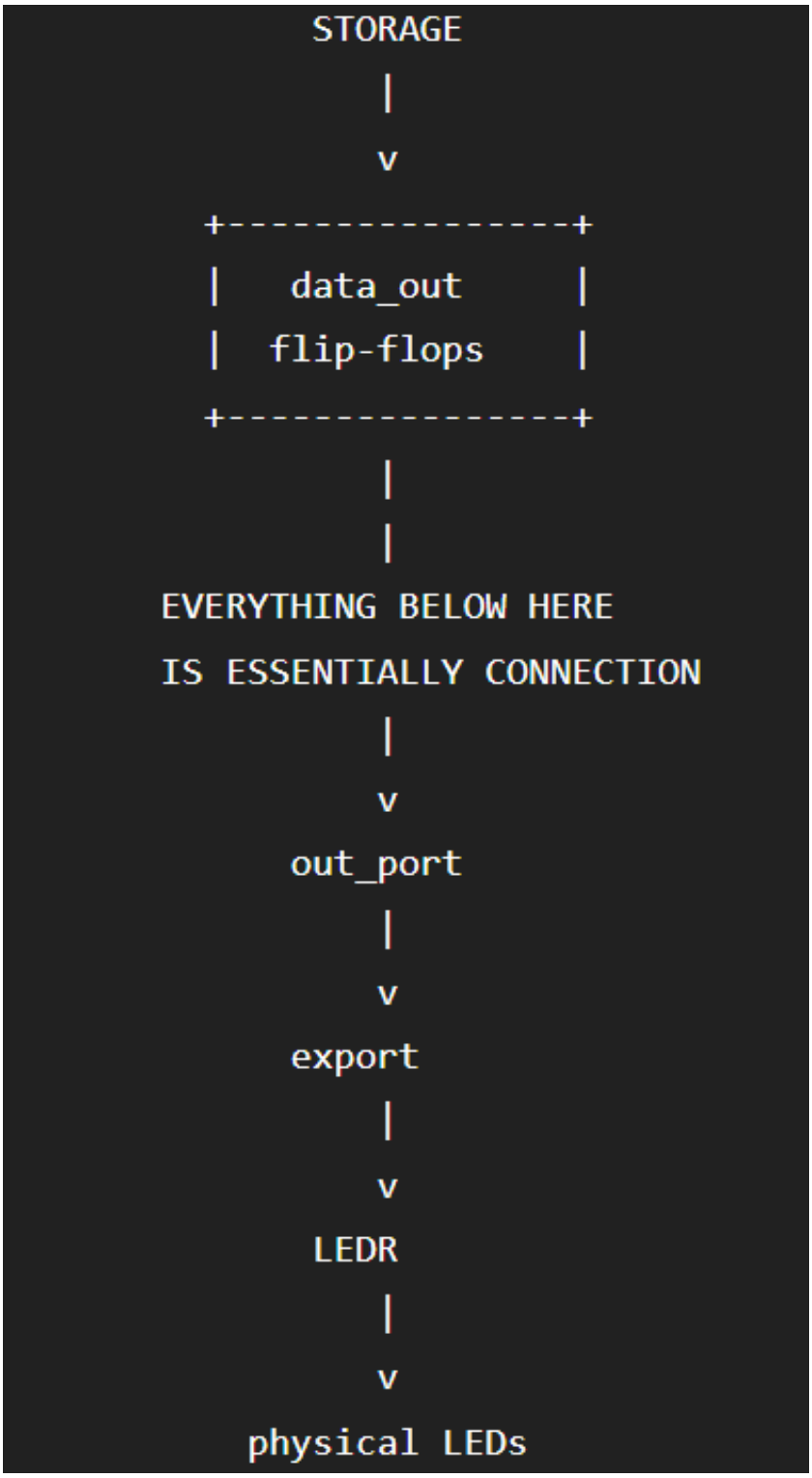
In particular, this signal is 10 bits wide because of how we set it in platform designer.

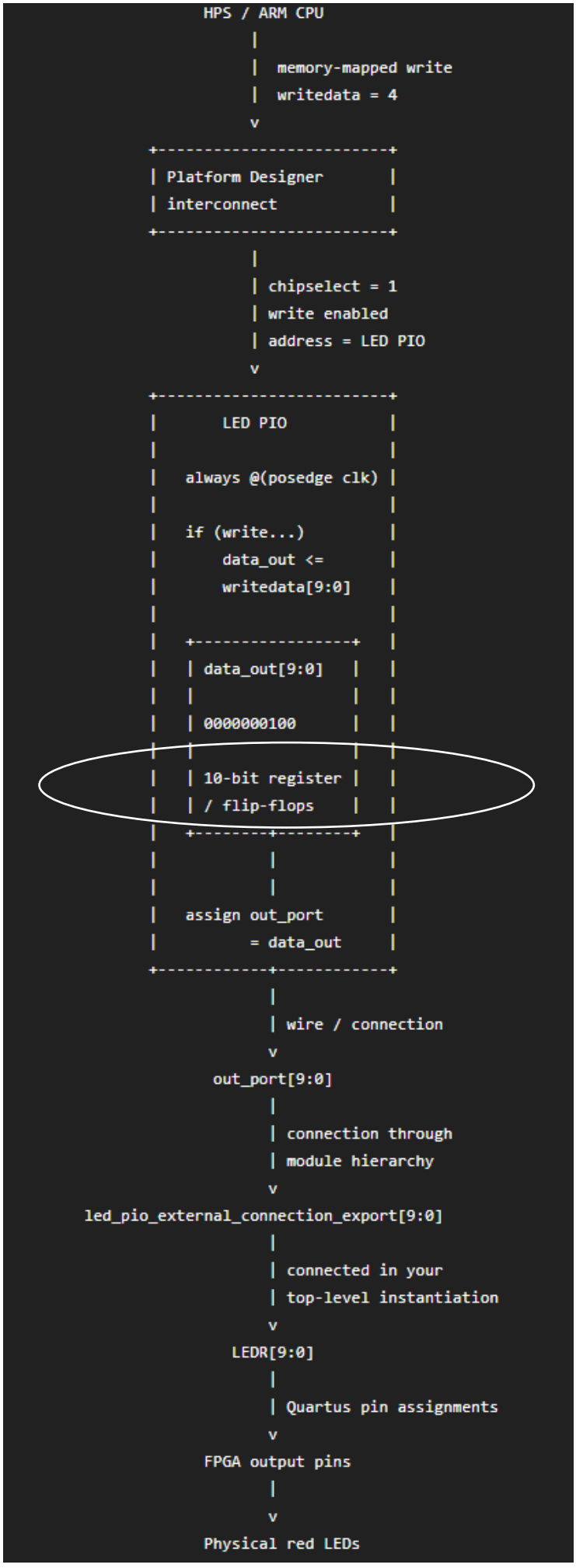
Finally, the out_port signal is connected to data_out.

```
module platform_designer_module_led_pio (
                                          // inputs:
                                           address,
                                           chipselect,
                                           clk,
                                           reset_n,
                                           write_n,
                                           writedata,

                                          // outputs:
                                           out_port,
                                           readdata
                                        )
;

  output  [  9: 0] out_port;
  output  [ 31: 0] readdata;
  input   [  1: 0] address;
  input            chipselect;
  input            clk;
  input            reset_n;
  input            write_n;
  input   [ 31: 0] writedata;


wire             clk_en;
reg     [  9: 0] data_out;
wire    [  9: 0] out_port;
wire    [  9: 0] read_mux_out;
wire    [ 31: 0] readdata;
  assign clk_en = 1;
  //s1, which is an e_avalon_slave
  assign read_mux_out = {10 {(address == 0)}} & data_out;
  always @(posedge clk or negedge reset_n)
    begin
      if (reset_n == 0)
          data_out <= 0;
      else if (chipselect && ~write_n && (address == 0))
          data_out <= writedata[9 : 0];
    end


  assign readdata = {32'b0 | read_mux_out};
  assign out_port = data_out;

endmodule
```

Moving up out from this block, we see that all the way up to the 10 bit wide LEDR signal we simply have connections.

ASCII art to represent this and then the big picture, with the C program as well:





Note that the only registers used are here.

This is important when thinking about what happens when the C program terminates. Since ctrl C out of the program does not rewrite a set of zeroes to the flip flop, the registers will hold the value of 4 indefinitely, and thus the LED stays lit indefinitely.